

CSCI 112

Programming with C

LECTURE: 19

J. L. Pach

OUTLINE:

- What is sorting and why do we need it?
- Bubble sort: the idea, a step-by-step trace, the implementation in C
- Quick sort: divide and conquer, the pivot, partitioning, recursion
- Complexity: why bubble sort is slow and quick sort is (usually) fast
- The built-in `qsort()` from `<stdlib.h>`
- The comparator: what it returns and **where these values come from** (`CMP` in assembly)
- Sorting your own data: `int`, `double`, strings and structures

SORTING

Basics

What is sorting?

Sorting means rearranging the elements of an array so that they follow a given **order**: ascending $(1, 2, 3, \ldots)$ or descending $(3, 2, 1, \ldots)$. For numbers the order is obvious; for text or structures **we** have to decide what "smaller" means (alphabetically? by age? by surname first?).

```
before:  5  1  4  2  8
after:   1  2  4  5  8      (ascending)
```

- Sorted data can be searched much faster (binary search: $O(\log n)$ instead of $O(n)$).
- Sorted data is easier to read, to compare and to merge.
- All sorting algorithms are built from just **two operations**: **compare** two elements and **swap** two elements.

Swapping two values

```c
void swap(int *a, int *b)
{
    int tmp = *a;
    *a = *b;
    *b = tmp;
}
```



- A function that has to change the caller's variables needs their **addresses** (pointers) – C passes arguments **by value**.
- We need a third, temporary variable. Without it the first assignment would destroy the original value of `*a`.
- Usage: `swap(&x, &y);` or `swap(&arr[i], &arr[j]);`

> Swapping is the only operation that **changes** the array. Everything else in a sorting algorithm just decides **which** two elements to swap.

Bubble sort: the idea

- Walk through the array and compare every **pair of neighbours**. If the left one is bigger than the right one – swap them.
- After the **first pass** the biggest element has "floated up" like a bubble to the end of the array, where it belongs. After the second pass the second biggest is in place, and so on.
- Each pass is therefore one element shorter: the tail of the array is already sorted and we do not touch it any more.
- We repeat until a pass makes **no swaps** – then the array is sorted.

Bubble sort: pass 1

Array: `5 1 4 2 8`. Brackets mark the pair being compared.

```
[5 1] 4  2  8    5 > 1  -> swap     1  5  4  2  8
 1 [5 4] 2  8    5 > 4  -> swap     1  4  5  2  8
 1  4 [5 2] 8    5 > 2  -> swap     1  4  2  5  8
 1  4  2 [5 8]   5 < 8  -> no swap   1  4  2  5  8
                                     ^
                          8 is in its final place
```

- Pass 1 made **4 comparisons** (n − 1) and the largest value, `8`, is at the end.
- Notice how `5` was carried along from the left, until it met a bigger neighbour (`8`).

Bubble sort: all passes

```
start:   5  1  4  2  8
pass 1:  1  4  2  5 |8      3 swaps, 4 comparisons
pass 2:  1  2  4 |5  8      1 swap,  3 comparisons
pass 3:  1  2  4  5  8      0 swaps, 2 comparisons  -> STOP
```

- `|` marks the border of the sorted tail.
- After pass 2 the array is already sorted, but the algorithm **does not know it yet** – it needs one more pass without any swap to be sure.
- This is why we remember whether a pass swapped anything (a `swapped` flag). On an already sorted array this gives just **one pass**: $O(n)$.

Bubble sort in C 1/2



```c
#include <stdbool.h>

void bubble_sort(int a[], int n)
{
    for (int pass = 0; pass < n - 1; pass++)
    {
        bool swapped = false;

        for (int i = 0; i < n - 1 - pass; i++)
        {
            if (a[i] > a[i + 1])
            {
                swap(&a[i], &a[i + 1]);
                swapped = true;
            }
        }
        if (!swapped)
        {
            break;              /* no swaps -> already sorted */
        }
    }
}
```

Bubble sort in C 2/2

```c
int main(void)
{
    int data[] = {5, 1, 4, 2, 8};
    int n = sizeof data / sizeof data[0];

    bubble_sort(data, n);

    for (int i = 0; i < n; i++)
    {
        printf("%d ", data[i]);
    }
    printf("\n");
    return 0;
}
```



- The inner loop stops at `n - 1 - pass`: the last `pass` elements are already in place.
- `a[i + 1]` must stay **inside** the array – this is why `i` goes only up to `n - 2` in the first pass.
- An array passed to a function **decays to a pointer**, so `sizeof` inside `bubble_sort` would not give the array size. That is why we pass `n` explicitly.

```
Result:
1 2 4 5 8
```

Bubble sort: complexity

- The outer loop runs up to $n - 1$ times, the inner loop up to $n - 1$ times: about $\frac{n(n-1)}{2}$ comparisons, i.e. $O(n^2)$.
- Doubling the data makes the sort about **4 times slower**. For $n = 1\,000\,000$ that is about $5 \cdot 10^{11}$ comparisons – far too many.
- It sorts **in place** (needs only one extra variable) and it is **stable** (equal elements keep their original order, because we swap only when `>`, never when `==`).

Case	Comparisons	When
Best	$O(n)$	already sorted (with the `swapped` flag)
Average	$O(n^2)$	random data
Worst	$O(n^2)$	sorted in reverse order

> Bubble sort is excellent for **learning** and useless for **real data**. Real programs use something faster.

QUICK

Sort

Quick sort: the idea

Quick sort uses the **divide and conquer** strategy: split a big problem into smaller ones, solve them in the same way, and the result is ready.

1. **Choose a pivot** – one element of the array (we will take the last one).
2. **Partition**: rearrange the array so that everything **smaller** than the pivot is on its left, and everything **bigger or equal** is on its right. The pivot lands in its **final** position.
3. **Recurse**: sort the left part and the right part in exactly the same way.
4. A part with 0 or 1 element is already sorted – this is the **base case** that stops the recursion.

```
[ smaller than pivot ]  pivot  [ bigger or equal ]
   sort recursively      done     sort recursively
```

Partition: a trace

Array `7 2 9 4 3 8 6`, pivot = last element = `6`. `i` marks the end of the "smaller" zone.

```
j=0:  7 >= 6          no move                 7 2 9 4 3 8 6
j=1:  2 <  6   i=0    swap a[0] <-> a[1]      2 7 9 4 3 8 6
j=2:  9 >= 6          no move                 2 7 9 4 3 8 6
j=3:  4 <  6   i=1    swap a[1] <-> a[3]      2 4 9 7 3 8 6
j=4:  3 <  6   i=2    swap a[2] <-> a[4]      2 4 3 7 9 8 6
j=5:  8 >= 6          no move                 2 4 3 7 9 8 6
end:  put pivot after the "smaller" zone:
      swap a[i+1] <-> a[high]                  2 4 3 6 9 8 7
                                       ^
                              pivot 6 is on its final index 3
```

⚒ Left part `2 4 3` – all smaller than `6`. Right part `9 8 7` – all bigger. They are still unsorted inside, so we repeat the procedure for both.

Partition in C

```c
int partition(int a[], int low, int high)
{
    int pivot = a[high];
    int i = low - 1;

    for (int j = low; j < high; j++)
    {
        if (a[j] < pivot)
        {
            i++;
            swap(&a[i], &a[j]);
        }
    }
    swap(&a[i + 1], &a[high]);
    return i + 1;
}
```



- `low` and `high` are the **indices** of the first and the last element of the part we work on.
- Elements in `a[low..i]` are **smaller** than the pivot, elements in `a[i+1..j-1]` are **bigger or equal**. `j` scans the rest.
- At the end the pivot is swapped into the gap between the two zones and its index is returned.

> This is the so-called **Lomuto partition scheme** – the simplest one to understand.

Quick sort in C

```c
void quick_sort(int a[], int low, int high)
{
    if (low < high)
    {
        int p = partition(a, low, high);
        quick_sort(a, low,   p - 1);
        quick_sort(a, p + 1, high);
    }
}
```

```c
int data[] = {7, 2, 9, 4, 3, 8, 6};
int n = sizeof data / sizeof data[0];
quick_sort(data, 0, n - 1);
```

- The condition `low < high` is the **base case**: a part with one element (or none) is sorted.
- The pivot at index `p` is already in place, so both recursive calls **skip** it: `p - 1` and `p + 1`.
- The first call covers the **whole** array: `0` and `n - 1` (the **last index**, not `n`!).

```
quick_sort(0,6) -> partition -> p=3
  quick_sort(0,2) [2 4 3]      quick_sort(4,6) [9 8 7]
```

Quick sort: complexity

- With a **good pivot** the array is split in halves: about $\log_2 n$ levels, and every level does about n comparisons in total. That gives $O(n \log n)$.
- With a **bad pivot** (the smallest or the biggest element every time) one part is empty and the other has $n - 1$ elements. It degenerates to $O(n^2)$ – just like bubble sort. With "the last element as pivot", this happens e.g. for an already sorted array.
- Better pivots: a random element, or the **median of three** (first, middle, last).
- Quick sort sorts **in place**, but it is **not stable**, and the recursion uses stack memory (about $\log n$ frames on average, up to n in the worst case).

Case	Time
Best / average	$O(n \log n)$
Worst	$O(n^2)$

Bubble sort vs quick sort

	Bubble sort	Quick sort
Idea	swap neighbours, biggest "floats up"	pivot + partition + recursion
Average time	$O(n^2)$	$O(n \log n)$
Worst time	$O(n^2)$	$O(n^2)$ (bad pivot)
Extra memory	$O(1)$	$O(\log n)$ – recursion stack
Stable	yes	no
For $n = 1\,000\,000$	about $5 \cdot 10^{11}$ comparisons	about $2 \cdot 10^7$ comparisons
Use in practice	learning only	the basis of library sorts

> Our `quick_sort` works **only for** `int`. To sort `double`, strings or structures we would have to copy and modify it every time. The standard library solves this problem – with `qsort()`.

BUILT-IN

qsort()

`qsort()` – the declaration

```c
#include <stdlib.h>

void qsort(void *base, size_t count, size_t size,
           int (*compar)(const void *, const void *));
```



Parameter	Meaning
`base`	address of the **first element** of the array (any type)
`count`	**how many** elements there are
`size`	size of **one element in bytes** – `sizeof arr[0]`
`compar`	pointer to **your function** that compares two elements

Why `void *` and a comparator?

C has no generics or templates. `qsort` does not know whether it is sorting `int`, `double` or your `struct`. It knows only **two things**: where the array starts and how many **bytes** one element takes. So it can **move** elements around (as raw memory, like `memcpy`), but it cannot tell which of two elements is smaller.

- That is why **you** give it a **comparator** – a function that knows the type and answers one question: *"which of these two elements goes first?"*
- `qsort` calls your function again and again, passing it **pointers to two elements** of the array (`const void *` because the type is unknown).
- Your function converts them back to the real type, compares the **values** and returns an `int`.

> Passing a function as an argument is done with a **function pointer**. The name of a function without `()` is its address: `qsort(arr, n, sizeof arr[0], compare_ints);`

What the comparator returns

For two elements `a` and `b` the comparator must return an `int`:

Returned value	Meaning	Order
negative (`< 0`)	`a` is **smaller** than `b`	`a` goes **before** `b`
zero (`== 0`)	`a` is **equal** to `b`	order does not matter
positive (`> 0`)	`a` is **bigger** than `b`	`a` goes **after** `b`

- Only the **sign** matters – the value does not have to be `-1`, `0`, `1`. It can be `-37` or `4096`.
- The same convention is used by `strcmp()` and `memcmp()` (see the previous lecture).
- Why exactly this convention? It is **not** an accident – it comes straight from the processor. See the next slides.

Where do these values come from? 1/3

Every processor must be able to answer: *is this number smaller, equal or bigger than that one?* The simplest way to find out is to **subtract** them:

```
a - b  <  0   ->   a  <  b     (e.g. 3 - 8  = -5)
a - b  == 0   ->   a  == b     (e.g. 5 - 5  =  0)
a - b  >  0   ->   a  >  b     (e.g. 9 - 2  =  7)
```

- We do not need to know the values themselves – the **sign of the difference** is enough to order any two numbers.
- Early C functions simply returned this difference, and the convention *"negative / zero / positive"* stayed in the whole language and the standard library.

Where do these values come from? 2/3

In **x86 assembly** the instruction `cmp` does exactly this: it **subtracts** the second operand from the first, **throws the result away** and keeps only the **flags** of the processor.

```
mov   eax, [a]
cmp   eax, [b]       ; computes eax - [b], sets flags, result discarded
jl    a_is_smaller   ; jump if less     (a - b < 0)
je    are_equal      ; jump if equal    (a - b = 0)
jg    a_is_bigger    ; jump if greater  (a - b > 0)
```

Flag	Name	Set when the result of `a - b` is…
`ZF`	Zero Flag	**zero** – the values are equal
`SF`	Sign Flag	**negative** – `a` is smaller than `b`

Where do these values come from? 3/3

The conditional jumps (`je`, `jl`, `jg`) only **read the flags** that `cmp` has just set. A C comparator is the same idea written in a high-level language: three possible answers – *smaller*, *equal*, *bigger*.

```
a - b       flags         jump    comparator returns
---------   -----------   -----   ------------------
negative    SF = 1          jl      negative  (a first)
zero        ZF = 1          je      0         (equal)
positive    ZF=0, SF=0      jg      positive  (b first)
```

- The real `jl` checks `SF != OF` (the **overflow flag** `OF` as well), because the subtraction of two big numbers can overflow and the sign alone would lie. The processor handles this correctly.
- **C does not.** A plain `a - b` written in your comparator **can overflow** – see the next slide.

Comparator for `int` 1/2

```c
/* tempting, but WRONG */
int cmp_bad(const void *a, const void *b)
{
    const int *pa = a;
    const int *pb = b;
    return *pa - *pb;
}
```



```c
/* INT_MAX - (-1) overflows
   -> undefined behaviour,
   typically a negative number,
   so "INT_MAX < -1"  */
```



- `*pa - *pb` copies the assembly idea directly, and for small numbers it works.
- For large values of opposite signs the result does **not fit in an** `int`: signed overflow is **undefined behaviour** in C, and in practice the sign is wrong – the sort produces garbage.
- The arguments are `const void *`: first convert them to `const int *`, then **dereference** to get the values.

Comparator for `int` 2/2

```c
/* ascending */
int cmp_asc(const void *a, const void *b)
{
    const int *pa = a;
    const int *pb = b;
    return (*pa > *pb) - (*pa < *pb);
}
```



```c
/* descending: swap the operands */
int cmp_desc(const void *a, const void *b)
{
    const int *pa = a;
    const int *pb = b;
    return (*pb > *pa) - (*pb < *pa);
}
```



- A comparison in C gives `1` (true) or `0` (false). So `(x > y) - (x < y)` gives:
  - `1 - 0 = 1` for `x > y`,
  - `0 - 0 = 0` for `x == y`,
  - `0 - 1 = -1` for `x < y`.
- There is **no subtraction of the values**, so there is no overflow.
- For **descending** order just swap the operands. Do not use `-cmp_asc(...)`: negating `INT_MIN` overflows too.

```c
qsort(data, n, sizeof data[0], cmp_asc);
```



Comparator for `double`

```c
int cmp_double(const void *a, const void *b)
{
    const double *pa = a;
    const double *pb = b;
    return (*pa > *pb) - (*pa < *pb);
}
```

```c
double v[] = {3.5, -1.2, 0.5, 2.0};
qsort(v, 4, sizeof v[0], cmp_double);
```

- **Never** write `return *pa - *pb;` for floating-point numbers. The function returns `int`, so the difference is **truncated**: `0.5 - 0.2 = 0.3` becomes `0` – "equal"!
- The `(x > y) - (x < y)` idiom works for any type that can be compared with `<` and `>`.

```
Result:
-1.2 0.5 2.0 3.5
```

Comparator for strings

```c
int cmp_str(const void *a, const void *b)
{
    const char *const *pa = a;
    const char *const *pb = b;
    return strcmp(*pa, *pb);
}
```

```c
const char *names[] = {"Kate", "Adam",
                       "Zoe",  "Bob"};
qsort(names, 4, sizeof names[0], cmp_str);
```

- The elements of the array are **pointers** (`char *`), so the comparator receives **pointers to pointers**. That is why there are two `*` in the type.
- `*pa` is the string itself (`char *`) – exactly what `strcmp()` wants.
- `strcmp()` already returns *negative / zero / positive* – it is a ready-made comparator for text. Include `<string.h>`.

```
Result:
Adam Bob Kate Zoe
```

> For a 2D array `char names[4][20]` an element is a whole row: the comparator can use `strcmp((const char *)a, (const char *)b)` directly, and `size` is `sizeof names[0]` (= 20).

Comparator for structures

```c
typedef struct
{
    char surname[20];
    int  age;
} Person;
```

```c
int cmp_person(const void *a, const void *b)
{
    const Person *pa = a;
    const Person *pb = b;

    int by_surname = strcmp(pa->surname,
                            pb->surname);
    if (by_surname != 0)
    {
        return by_surname;
    }
    return (pa->age > pb->age) - (pa->age < pb->age);
}
```

- Compare by the **first key**. If it **differs**, that is the answer – return it immediately.
- Only when the first keys are **equal** (the result is `0`), compare by the **second key**, and so on.
- This gives the order: by surname, and people with the same surname by age.
- To sort by another field, write **another comparator** – the data stays the same.

Using `qsort()` – a full example

```
Person people[] = { {"Smith", 40}, {"Brown", 25}, {"Smith", 22}, {"Adams", 31} };
int n = sizeof people / sizeof people[0];

qsort(people, n, sizeof people[0], cmp_person);

for (int i = 0; i < n; i++)
{
    printf("%-8s %d\n", people[i].surname, people[i].age);
}
```



```
Result:
Adams    31
Brown    25
Smith    22
Smith    40
```

To sort your own data you need exactly three things: an array and the number of its elements, the size of one element (`sizeof arr[0]`), and a comparator `int f(const void *, const void *)` that returns negative / zero / positive.

Common mistakes

- Passing `sizeof arr` (the size of the **whole** array) instead of `sizeof arr[0]` (one **element**).
- Using `a - b` for `int` (overflow) or for `double` (truncation).
- Forgetting that the comparator gets **pointers to elements**: for `int` it is `const int *`, for `char *` elements it is `const char *const *`.
- Returning only `0` and `1` (e.g. `return *pa > *pb;`). The comparator **must** be able to return a negative number as well.
- A comparator that is **inconsistent** (`a < b` and also `b < a`) gives undefined behaviour of `qsort`.
- Assuming that `qsort` is **stable**. If you need that, add a second key (e.g. the original index) to the comparator.
- Calling `qsort` on a pointer that does not point to the array of `count` elements – `qsort` does not check anything.

THANK

You!